# Screen-recording script

LifeLog CI/CD walkthrough, about 20 minutes. Green boxes are what you say: read them naturally in your own words, do not memorise them. **Show** is what is on screen, **Type** is what you type.

## Before you press record

Do this once, 10 minutes before. A calm recording starts with a calm desk.

| Check | How |
|---|---|
| EC2 instance is **Running** | AWS Console, EC2, Instances. If you stopped it, start it: the public IP changes, so note the new one. |
| Minikube is up | On the instance: `sudo -u jenkins -H minikube status`. If not Running: `sudo -u jenkins -H minikube start` |
| Port-forward is running | `curl -s localhost:30080/health` on the instance. If empty, start it (command below). |
| Deployed app is the **version without Goals** | Open `http://<EC2-IP>:30080`. The nav must show Today, Log, Insights, Trackers only. |
| Jenkins is logged in and has at least one green build | `http://<EC2-IP>:8080` |
| Mac repo is clean on `main` | `cd ~/lifelog-cicd && git checkout main && git pull && git status -sb` |
| No secrets on screen | Close the Docker Hub token page. Do not open the `dockerhub-creds` credential. |
| Screen is readable | Browser zoom 125 percent (Cmd and +), terminal font size 16 or more, Do Not Disturb on, bookmarks bar hidden. |

Start the port-forward on the instance if needed:

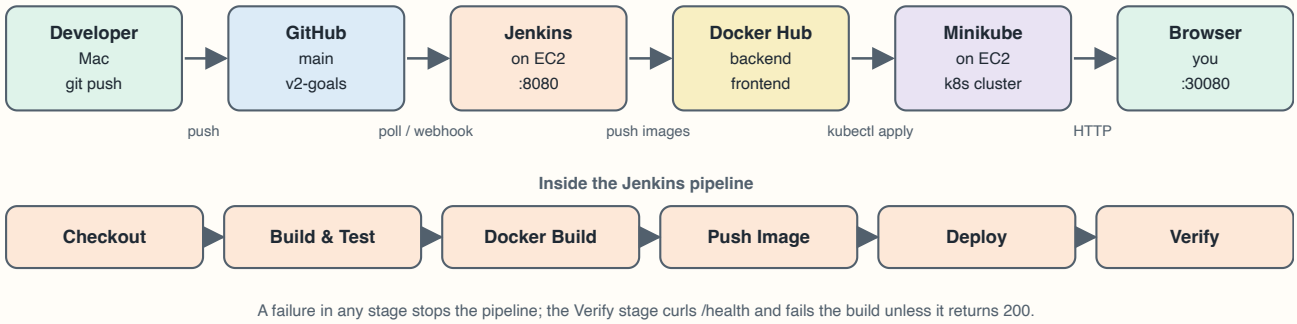
```
sudo pkill -f "kubectl port-forward"
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/lifelog 30080:80 >
/tmp/portforward.log 2>&1 &
```

**Tabs, left to right:** GitHub repo, AWS EC2 console, Jenkins, Docker Hub, the app. **Terminals:** Mac in `~/lifelog-cicd`, SSH to the instance, and a second SSH tab for `kubectl get pods -w`. Record with **Cmd+Shift+5, Record Entire Screen**, microphone set to your Mac's mic.

## Timeline

| # | Section | Minutes |
|---|---|---|
| 1 | Introduction | 1:00 |
| 2 | The app, live | 2:00 |
| 3 | The repository | 1:30 |
| 4 | Tests | 0:40 |
| 5 | Docker: two images | 2:00 |
| 6 | AWS | 1:30 |
| 7 | Jenkins setup | 1:30 |
| 8 | The Jenkinsfile | 2:30 |

| # | Section | Minutes |
|---|---|---|
| 9 | Running the pipeline | 1:30 |
| 10 | Docker Hub | 0:45 |
| 11 | Kubernetes | 2:00 |
| 12 | Second run: v2-goals | 3:00 |
| 13 | Wrap-up | 1:00 |




## 1. Introduction (1:00)

**Show:** the app in the browser at `http://<EC2-IP>:30080`, then scroll to the footer.

> Hello, I am Saksham. This is my CCA 2 project: a complete CI/CD pipeline for a web app called LifeLog. A developer pushes code to GitHub, Jenkins builds and tests it, creates Docker images, pushes them to Docker Hub, and deploys them to a Kubernetes cluster running on AWS. All of it happens automatically, with no manual steps after the push.
>
> The footer of every page shows the version, build number, git commit and the names of the pods serving me. That lets me prove, in the browser, which pipeline run is live right now.

## 2. The app, live (2:00)

**Show:** the Today page. Press `/` to jump to the capture bar.

**Type, one at a time, pressing Enter after each. Pause on the preview before Enter:**

```
wat 3
spent 250 lunch
yesterday gym
pushups 20
undo
```

> LifeLog is a one-line life log. Instead of forms, there is a capture bar at the top of every page. I press slash to focus it and type plain text. Before I press Enter it shows a live preview of what will happen. Here "wat 3" matched my Water tracker by prefix and will add three glasses.
>
> "Spent 250 lunch" is recognised as money, and the category Food is inferred from the word lunch. "Yesterday gym" backfills yesterday. "Pushups" is not a tracker yet, so the bar offers a one-click button to create it. And "undo" reverses the last action.

**Show:** click through **Log**, **Insights**, **Trackers** (open one tracker to show the heatmap).

> Everything I captured appears in the Log. Insights shows streaks, trends and spending charts, and every tracker has a heatmap of the last twelve weeks. The parser is pure Python with no database access, which is why it is easy to test.

## 3. The repository (1:30)

**Show:** the GitHub repo page. Click into `app/`, `tests/`, `backend/`, `frontend/`, `k8s/`, then open the branch dropdown.

> The code is on GitHub. The app folder holds the Flask backend, the capture parser, the statistics and the service layer. Tests are in the tests folder. For DevOps I have a Dockerfile for the backend, a separate folder for the frontend image, the Kubernetes manifests in k8s, and the Jenkinsfile at the root.
>
> There are two branches. Main is version 1. The v2-goals branch adds a Goals module. I will use that for the second run at the end, to show a code change flowing automatically all the way to the cluster.

## 4. Tests (0:40)

**Show:** the Mac terminal.

**Type:**

```
cd ~/lifelog-cicd
.venv/bin/pytest -q | tail -3
```

> The project has 65 automated tests covering the parser, streak logic, statistics, the API and every page. Jenkins runs these same tests as its first quality gate. If any test fails, nothing is built or deployed.

## 5. Docker: two images (2:00)

**Show:** open `backend/Dockerfile`, then `frontend/Dockerfile` and `frontend/nginx.conf` in the GitHub tab or your editor.

> I split the app into two images. The backend image is Python 3.12 slim with Flask and gunicorn. It runs as a non-root user, has a health check, and accepts build arguments for the version, git commit and build number, which become environment variables. That is where the footer gets its values.
>
> The frontend image is nginx. It serves the CSS and JavaScript directly and forwards every other request to the backend. nginx also adds a header with its own pod name, which the footer displays.

**Type:**

```
docker compose up -d --build
docker images | grep lifelog
curl -s localhost:8080/health
docker compose down
```

> Docker Compose builds and runs both containers together on my laptop. This confirms the images work before the pipeline ever touches them.

## 6. AWS (1:30)

**Show:** AWS Console, EC2, the instance; then the **Security** tab.

> The pipeline runs on one EC2 server in Mumbai: Ubuntu 24.04, 2 CPUs and 8 GB of RAM. I used a free-plan eligible instance type with enough memory, because Jenkins and Minikube together need around 4 GB.
>
> The security group controls who can connect. Port 22 is for SSH, 8080 is Jenkins, and 30080 is the application. All three are restricted to my own IP address, so the server is not open to the internet.

**Show:** the SSH terminal. **Type:**

```
java –version 2>&1 | head –1
docker ––version
kubectl version ––client | head –1
minikube version | head –1
systemctl is–active jenkins
```

> On the server I installed Java 21, Docker, Jenkins, kubectl and Minikube. Jenkins now requires Java 21, which I found when it refused to start on Java 17.

## 7. Jenkins setup (1:30)

**Show:** Jenkins dashboard, **Manage Jenkins, Plugins, Installed**, then **Credentials**, then the job's **Configure** page.

> Jenkins is the automation server. Besides the suggested plugins I installed Docker Pipeline and Pipeline Stage View, which gives the stage-by-stage picture you will see in a moment.
>
> To push images, Jenkins needs to log in to Docker Hub. I stored a Docker Hub access token as a credential called dockerhub-creds. I used a token instead of my password because it can be revoked and limited to what it needs, and Jenkins never shows its value in logs.
>
> The job is a pipeline from source control. It points at my GitHub repository, branch main, and reads the file called Jenkinsfile. Two setup details matter. The jenkins user is in the docker group so it can build images, and Minikube was started as the jenkins user so the pipeline's kubectl commands can reach the cluster.

## 8. The Jenkinsfile (2:30)

**Show:** the `Jenkinsfile` on GitHub, scrolling stage by stage as you speak.

> The whole pipeline is code in this one file. There are six stages.
>
> **Checkout** pulls the code and records the short commit id. **Build and Test** creates a Python virtual environment, installs the pinned requirements and runs pytest. A failure here stops everything.
>
> **Docker Build** builds both images with the build number, commit and version as build arguments, and tags each one with the build number and also latest. **Push Image** logs in with the stored credential, using password-stdin so the secret never appears on a command line or in the log, and pushes both images.
>
> **Deploy to Minikube** is the interesting one. The manifests in git say latest, so Jenkins rewrites the image line in its own copy to the exact build number, runs kubectl apply, and waits for both rollouts to finish. That makes every deployment pinned to one build.
>
> **Verify** lists the pods, deployments and services, then calls the health endpoint through the NodePort using minikube ip, and fails the build unless it gets HTTP 200. In the post section, Jenkins always logs out of Docker Hub, and on a failure it prints the Kubernetes description and logs to help debugging. There is also a poll trigger every five minutes as a fallback when no webhook can reach the server.

## 9. Running the pipeline (1:30)

**Show:** Jenkins, the job page. Click **Build Now**, then open the running build and **Console Output** alongside **Stage View**.

> I will trigger a run now. The Stage View fills in as each stage completes. The whole thing takes about a minute.

**Point at, in the console:** the pytest line, the `docker build` lines for backend and frontend, `Login Succeeded`, `successfully rolled out`, and the health JSON at the end.

> Here are the tests passing, here are both images being built, the login to Docker Hub, the push, and the rollout of the backend and the frontend. The last stage hits the health endpoint, gets status ok, and the build finishes with SUCCESS.

## 10. Docker Hub (0:45)

**Show:** Docker Hub, your repositories, then the **Tags** tab of `lifelog-backend`.

> Here are the two images the pipeline just pushed. Every build has its own tag, the build number, plus latest. If a release is bad, I can redeploy an older tag, so the tags double as a version history.

## 11. Kubernetes (2:00)

**Show:** the SSH terminal. **Type:**

```
sudo -u jenkins -H kubectl get pods,deploy,svc
sudo -u jenkins -H kubectl describe deploy lifelog-backend | grep -E
"Replicas|Image|Liveness|Readiness|Limits|Requests" -A1
curl -s localhost:30080/health
```

> Kubernetes is running two frontend pods and one backend pod. The backend is a single replica on purpose: it stores data in a SQLite file inside the pod, and two copies would each have their own separate data. The frontend is stateless, so two replicas are safe and give availability.
>
> Outside the cluster, only one thing is exposed: the NodePort service on 30080. Inside, nginx reaches the backend through an internal service called backend. The deployment has readiness and liveness probes on the health endpoints, CPU and memory limits, and it uses the downward API to inject each pod's own name, which the footer shows.
>
> One limitation I want to explain. Minikube with the Docker driver runs inside its own network on the server, so port 30080 is not reachable from outside. That is why the pipeline checks it from the server, and for my browser I use a port-forward on all interfaces, plus the open port in the security group.

## 12. Second run: v2-goals (3:00)

**Show:** the Mac terminal first, the app second, Jenkins third.

> Now the real point of CI/CD: shipping a change. The v2-goals branch adds a Goals page. I will merge it into main and push, and I will not touch the server.

**Type on the Mac:**

```
cd ~/lifelog-cicd
git checkout main && git pull
git merge --no-ff v2-goals -m "Merge v2-goals: add Goals module"
git push origin main
```

**Show:** Jenkins. Click **Build Now** (or wait for the five-minute poll). In the second SSH tab, **type:**

```
sudo -u jenkins -H kubectl get pods -w
```

> The pipeline starts, runs all 74 tests, builds new images tagged with this build number, pushes them, and updates the cluster. Watch the pods: new ones start, and only when they pass their readiness probe do the old ones terminate. That is a rolling update, so the site stays up the whole time.

**When the build is green, press Ctrl+C, then restart the port-forward:**

```
sudo pkill -f "kubectl port-forward"
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/lifelog 30080:80 >
/tmp/portforward.log 2>&1 &
```

> A port-forward is attached to one specific pod, so after a rollout I restart it. In a real setup an ingress or load balancer would remove that step.

**Show:** the browser. Hard refresh with **Cmd+Shift+R**, click **Goals**, add a goal, then scroll to the footer.

> The Goals link has appeared, the footer shows the new version and a higher build number, and both pod names are different, because Kubernetes replaced them. The code went from a merge on my laptop to production with no manual deployment steps.

## 13. Wrap-up (1:00)

**Show:** the Jenkins Stage View with both runs.

> To summarise: GitHub holds the code, Jenkins tests, builds and pushes two Docker images, Docker Hub stores them with a tag per build, and Kubernetes on AWS runs them with health checks and rolling updates.
>
> Limitations and next steps: data lives in the pod, so I would add a persistent volume or a real database. I would add an ingress, a webhook with a proper network path, vulnerability scanning of the images, and separate staging and production. And I stop the instance after the demo to save credits.
>
> Thank you.

## If something goes wrong while recording

| Problem | Fix, and what to say |
|---|---|
| Page does not load | Your IP changed: update the security group rules to **My IP**. Say: "the security group only allows my IP." |
| Page loads but is the old design | Hard refresh with Cmd+Shift+R. |
| Port 30080 refused | The port-forward died after a rollout: run the restart command. |
| Build fails | Open Console Output and read the last 20 lines. Failures are part of CI/CD: say which stage stopped the pipeline and why that is a good thing. |
| `minikube` or `kubectl` cannot connect | `sudo –u jenkins –H minikube start` |
| Webhook shows a red cross on GitHub | Expected: port 8080 is closed to GitHub. Say: "the poll trigger is the fallback." |